

The Vector-Space Model

- Assume t distinct terms remain after preprocessing; call them index terms or the vocabulary.
- These “orthogonal” terms form a vector space.

Dimension = $t = |\text{vocabulary}|$

- Each term, i , in a document or query, j , is given a real-valued weight, w_{ij} .
- Both documents and queries are expressed as t -dimensional vectors:

$$d_j = (w_{1j}, w_{2j}, \dots, w_{tj})$$

Document Collection

- A collection of n documents can be represented in the vector space model by a term-document matrix.
- An entry in the matrix corresponds to the “weight” of a term in the document; zero means the term has no significance in the document or it simply doesn’t exist in the document.

$$\begin{pmatrix} & T_1 & T_2 & \dots & T_t \\ D_1 & w_{11} & w_{21} & \dots & w_{t1} \\ D_2 & w_{12} & w_{22} & \dots & w_{t2} \\ \vdots & \vdots & \vdots & & \vdots \\ \vdots & \vdots & \vdots & & \vdots \\ D_n & w_{1n} & w_{2n} & \dots & w_{tn} \end{pmatrix}$$

Term Weights: Term Frequency

- More frequent terms in a document are more important, i.e. more indicative of the topic.

f_{ij} = frequency of term i in document j

- May want to normalize *term frequency* (tf) by dividing by the frequency of the most common term in the document:

$$tf_{ij} = f_{ij} / \max_i \{f_{ij}\}$$

Term Weights: Inverse Document Frequency

- Terms that appear in many *different* documents are *less* indicative of overall topic.

df_i = document frequency of term i

= number of documents containing term i

idf_i = inverse document frequency of term i ,

= $\log_2 (N / df_i)$

(N : total number of documents)

- An indication of a term's *discrimination* power.
- Log used to dampen the effect relative to tf .

TF-IDF Weighting

- A typical combined term importance indicator is *tf-idf* weighting:

$$w_{ij} = tf_{ij} idf_i = tf_{ij} \log_2 (N / df_i)$$

- A term occurring frequently in the document but rarely in the rest of the collection is given high weight.
- Many other ways of determining term weights have been proposed.
- Experimentally, *tf-idf* has been found to work well.

Computing TF-IDF -- An Example

Given a document containing terms with given frequencies:

A(3), B(2), C(1)

Assume collection contains 10,000 documents and document frequencies of these terms are:

A(50), B(1300), C(250)

Then:

A: $tf = 3/3$; $idf = \log_2(10000/50) = 7.6$; $tf-idf = 7.6$

B: $tf = 2/3$; $idf = \log_2(10000/1300) = 2.9$; $tf-idf = 2.0$

C: $tf = 1/3$; $idf = \log_2(10000/250) = 5.3$; $tf-idf = 1.8$